

Sonora — Documentação da Base de Dados (v2)

1. Visão Geral

A base de dados é fornecida pela plataforma Base44. Cada entidade é um schema JSON em `base44/entities/<Name>.jsonc` que define propriedades e regras de Row-Level Security (RLS). Todos os registos têm campos built-in: `id` (string), `created_date` (ISO), `updated_date` (ISO), `created_by_id` (id do criador), `created_by` (email do criador). Não se declaram estes campos no schema. As operações são feitas via SDK: `list`, `filter`, `get`, `create`, `update`, `bulkCreate`, `bulkUpdate`, `updateMany`, `delete`, `deleteMany`, `schema`, `subscribe`.

2. Song (catálogo)

Propriedades: `title` (string, required), `artist_id` (ref Artist, required), `artist_name` (string, denormalizado), `album_id` (ref Album), `album_title` (string, denormalizado), `cover_url` (string), `duration_seconds` (number, default 0), `audio_url` (string), `genre` (string), `play_count` (number, default 0), `track_number` (number, default 1), `release_year` (number). Sem RLS (público a autenticados). Usado em todo o lado: filas, recomendações, favoritos, histórico. A denormalização (`artist_name`, `album_title`, `cover_url`) evita joins: cada Song já traz o necessário para renderizar um SongRow sem queries extra.

3. Album (catálogo)

Propriedades: `title` (string, required), `artist_id` (ref Artist, required), `artist_name` (string), `cover_url` (string), `release_year` (number), `genre` (string), `track_count` (number, default 0). Sem RLS. AlbumDetail carrega `Album.get(id) + Song.filter({album_id: id}, 'track_number', 100) + Playlist.list` (para as tags de playlist).

4. Artist (catálogo)

Propriedades: `name` (string, required), `bio` (string), `image_url` (string), `header_image_url` (string), `genres` (array de strings), `monthly_listeners` (number, default 0). Sem RLS. ArtistDetail mostra bio, capa (`header_image_url`), géneros, álbuns (`Album.filter({artist_id})`) e músicas (`Song.filter({artist_id})`). `playArtistShuffle` usa `getArtistShuffleQueue(artist_id)`.

5. Podcast e PodcastEpisode (catálogo)

Podcast: `title` (required), `author` (required), `cover_url`, `description`, `category`. Sem RLS. PodcastEpisode: `title` (required), `podcast_id` (ref Podcast, required), `podcast_title`, `duration_seconds`, `audio_url`, `description`, `publish_date` (date), `cover_url`. Sem RLS. A página Podcasts integra com a iTunes Search API para descobrir novos podcasts e subscreve (cria Podcast + episódios) na base de dados local.

6. Playlist

Propriedades: `title` (string, required), `description` (string), `cover_url` (string), `song_ids` (array de strings = ids de Song), `is_liked` (bool, default false), `is_public` (bool, default true), `track_count` (number, default 0). RLS: `read = { $or: [ { data.is_public: true }, { created_by_id: {{user.id}} } ] }` — públicas visíveis a todos, privadas só ao dono. `create: true`. `update/delete: { created_by_id: {{user.id}} }` — só o dono edita/remove. O `track_count` é mantido sincronizado manualmente nos writes (`AddToPlaylistDialog`, `PlaylistDetail`). A sidebar subscreve alterações (realtime) para refletir criações/edições.

7. Favorite (pessoal)

Propriedades: `song_id` (string, required), `song_title`, `artist_name`, `cover_url`, `added_at` (date-time). RLS: `read/update/delete = { created_by_id: {{user.id}} }` — cada utilizador só vê e gere os seus. `create: true`. Usado para a secção Favoritos da biblioteca e para o botão coração nas músicas (`SongRow.handleLove` cria/apaga por `song_id` via `deleteMany({song_id})`).

8. ListeningHistory (pessoal)

Propriedades: `song_id` (required), `song_title`, `artist_name`, `cover_url`, `played_at` (date-time), `play_duration` (number, default 0). RLS: `read/update/delete = { created_by_id: {{user.id}} }` — só o dono vê o seu histórico. O `PlayerContext` cria um registo por reprodução (no efeito de mudança de música). Usado para 'Recentes' na biblioteca e para a página Estatísticas (top artistas/géneros/músicas por contagem).

9. Friendship (social — convites e amizades)

Propriedades: `addressee_email` (string, required), `addressee_id` (string, ref User), `status` (enum `pending|accepted|declined`, default `pending`, required). RLS `read/update/delete = { $or: [ { created_by_id: {{user.id}} }, { data.addressee_email: {{user.email}} }, { data.addressee_id: {{user.id}} } ] }` — ambas as partes veem e gerem. `create: true`. Um convite `pending` torna-se `accepted` quando o destinatário faz `update` com `addressee_id = o seu id`. A lista de amigos obtém-se filtrando `status='accepted'` e determinando o 'amigo' = a outra parte (se `created_by_id == me ! addressee_id`, senão `created_by_id`). Para `base44.asServiceRole.entities.Friendship.create` para definir `created_by_id` diferente do utilizador atual (simulando um convite recebido).

10. FriendActivity (social — atividade de escuta)

Propriedades: `song_id`, `song_title` (required), `artist_name`, `cover_url`, `is_playing` (bool, default `true`). RLS: `read = {}` (aberto a todos os autenticados — para os amigos verem); `create/update/delete = { created_by_id: {{user.id}} }` — só o dono escreve. O `PlayerContext` cria um registo por reprodução (se `share_activity != false`) e atualiza `is_playing` em `pause/play`. Mantém-se máximo 20 por utilizador: após criar, busca os 30 mais recentes e apaga os excedentes com `deleteMany({ id: { $in: ids } })`. A página Amigos mostra o registo mais recente por amigo (agrupar por `created_by_id`, sort -`created_date`).

11. Profile (social — perfil público)

Propriedades: `display_name` (string, required), `photo_url`, `cover_photo_url`, `bio`. RLS: `read = {}` (público a autenticados); `create/update/delete = { created_by_id: {{user.id}} }`. Cada utilizador tem um registo `Profile`, sincronizado a partir do `ProfileModal` (que também faz `updateMe` no `User`). Permite que amigos vejam nome/foto/capa/bio sem precisar de listar `Users` (o `User` built-in só é listável por admins).

12. User (built-in)

Gerido pela plataforma. Campos: `id`, `created_date`, `full_name`, `email`, `role` (`admin|user`). Campos custom adicionados via `updateMe`: `bio`, `cover_photo_url`, `share_activity` (bool, default `true` — toggle de privacidade da atividade). Não se podem criar/importar registos `User` (`create` retorna 405); os utilizadores entram por convite (`base44.users.inviteUser(email, role)`). Só admins listam/editam/apagam outros utilizadores. Não se adiciona RLS ao `User` (a plataforma gere as permissões).

13. Relacionamentos (refs lógicas)

`Song.artist_id !' Artist.id; Song.album_id !' Album.id; Album.artist_id !' Artist.id; Podcast.id; Playlist.song_ids[] !' Song.id[]; Favorite.song_id !' Song.id; Friendship.addressee_id !' User.id; FriendActivity.song_id !' Song.id; Profile.` foreign keys enforçadas — a integridade é gerida pela aplicação. Denormaliza-se frequentemente (`artist_name`, `album_title`, `cover_url`) para evitar joins e tornar os `SongRows` autossuficientes.

14. Populate / Seed

Os dados de catálogo (`Song`, `Album`, `Artist`, `Podcast`, `PodcastEpisode`) são populados via `create_entity_records` ou `import_data` (CSV/Excel/JSON). Exemplo: `create_entity_records('Song', [{title, artist_id, artist_name, audio_url, cover_url, duration_seconds, track_number, genre, play_count}])`. Para `Friendship` de teste, usou-se `base44.asServiceRole.entities.Friendship.create` para definir `created_by_id` diferente do utilizador atual. Os utilizadores não se populam — entram por convite. O cache de músicas (`songsCache` em `recommendations.js`) deve ser limpo (`clearSongsCache`) após popular novas músicas para que as recomendações as incluam.

15. Consultas Típicas (SDK)

`list(sort, limit): Playlist.list('-updated_date', 100). filter(query, sort, limit): Song.filter({album_id: id}, 'track_number', 100); FriendActivity.filter({created_by_id: id}, '-created_date', 5); Friendship.filter({status:'accepted'}). get(id): Album.get(id). create(data): Playlist.create({...}). update(id, data): Playlist.update(id, {song_ids, track_count}). bulkCreate([]), bulkUpdate([]). updateMany(query, {$set:...}) — mesmo change a todos os matches (até 500; o $set deve mudar um campo do query para não re-match infinito). deleteMany(query): Friendship.deleteMany({status:'declined'}) — cuidado com queries largas. subscribe(cb): realtime. schema(): JSON schema para formulários.`

16. Segurança e Notas

RLS é aplicada no servidor em cada pedido de app-user. Operações sem regra ficam abertas — por isso dados pessoais têm read/update/delete restritos a `created_by_id`. Writes usam formas compactas (`equality`, `$in`, `$or/$and`). O `created_by_id` é o anchor mais fiável (sempre presente). Para `FriendActivity/Profile` o read é aberto intencionalmente (partilha social), mas os writes são só do dono. Não há triggers de DB custom; a lógica de limpeza (máx 20 `FriendActivity`) e de escrita de histórico vive no `PlayerContext` (cliente). O `deleteMany` deve ter queries específicas para evitar apagar dados alheios.